

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Software

Asignatura
Aplicaciones Distribuidas

Actividad
FORM-FB-01: Fichas bibliográficas sobre fundamentos de sistemas distribuidos y arquitecturas distribuidas

Docente
Ing. Gleiston Cicerón Guerrero Ulloa

Estudiante
Zambrano Yong Angel Daniel

Curso
Séptimo nivel, paralelo B

Repositorio
`https://github.com/Kinetro/QZ-Practicas_Experimentales/tree/main/Semana%202/FORM-FB-01`

Año lectivo 2026–2027
Quevedo – Los Ríos – Ecuador
8 de noviembre de 2026

Presentación

Estas fichas reúnen las fuentes que usaré en el informe TA-IND-01, donde analizaré Apache Cassandra y lo compararé con MongoDB. Escogí tres artículos experimentales que miden el costo de la consistencia en bases NoSQL, una revisión sobre los compromisos de la replicación (CAP y PACELC) y otra sobre consenso tolerante a fallos. Todas las fuentes son de 2021 a 2025.

Cada ficha tiene la referencia en formato IEEE, generada con biblatex desde el archivo `.bib`, el DOI, un resumen, un dato concreto con su ubicación y una nota personal.

Ficha 1: Niveles de consistencia en bases NoSQL distribuidas en la nube

Referencia (IEEE): S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Benchmarking consistency levels of cloud-distributed NoSQL databases using YCSB," *IEEE Access*, vol. 13, pp. 63428–63438, 2025. DOI: 10.1109/ACCESS.2025.3558923

Identificador: DOI: 10.1109/ACCESS.2025.3558923

Tipo de fuente: Artículo de revista indexada (IEEE Access)

Resumen

Es un estudio experimental que mide cuánto rendimiento pierden Cassandra, MongoDB y Redis cuando se les exige consistencia fuerte. Los trabajos anteriores solían evaluar una sola base o dejaban fuera las réplicas repartidas en varias regiones. Montaron en Google Cloud un nodo primario y dos réplicas, cada uno en una región distinta de Estados Unidos, y corrieron las cargas de YCSB con consistencia débil y fuerte, entre 10 mil y 100 mil operaciones. Con consistencia fuerte las tres bases procesan muchas menos operaciones por segundo y tardan bastante más. Redis es la más rápida, pero también la que más cae al cambiar de nivel.

Dato concreto

Promediando todas las cargas, pasar de consistencia débil (ONE) a fuerte (ALL) bajó el throughput de Cassandra en 64,3 % y subió su tiempo de ejecución en 700,71 %. En MongoDB las cifras fueron 60,49 % y 719,8 %, así que en promedio las dos pagan un precio parecido.
Ubicación: Sección V-B (Performance summary) y Tabla 4, p. 63434.

Nota personal

No esperaba que el costo promedio fuera casi igual en Cassandra y en MongoDB; pensaba que Cassandra, al estar diseñada para priorizar la disponibilidad, iba a sufrir mucho más. Voy a usar este dato en la tabla comparativa del TA-IND-01, en la fila de rendimiento con consistencia fuerte. También me sirve para no exagerar en el informe, porque la diferencia entre las dos aparece en cargas puntuales, como la D, donde Cassandra cae 75 % y MongoDB 64 %.

Ficha 2: Impacto de la consistencia en NoSQL georeplicado

Referencia (IEEE): S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications," *Journal of Cloud Computing*, vol. 13, no. 1, 2024, Art. no. 158. DOI: 10.1186/s13677-024-00716-7

Identificador: DOI: 10.1186/s13677-024-00716-7

Tipo de fuente: Artículo de revista indexada (Journal of Cloud Computing, Springer)

Resumen

Es un experimento de los mismos autores de la ficha 1 sobre cómo cambia el rendimiento de Cassandra, MongoDB y Redis según el nivel de consistencia cuando las réplicas están lejos entre sí. Querían medir el efecto de la distancia entre nodos, que casi no se había estudiado. Desplegaron las bases en Google Cloud en una sola región y en varias regiones, y generaron carga con Locust de 1 a 10 usuarios concurrentes, en lecturas y escrituras. En una región, la consistencia fuerte afecta sobre todo las lecturas de Cassandra. Con varias regiones todas pierden mucho, y las escrituras de MongoDB son las más golpeadas.

Dato concreto

En el despliegue multirregión, las escrituras de Cassandra bajaron de unas 1300 peticiones por segundo con consistencia débil a unas 25 con consistencia fuerte, una caída del 98 % con 2 usuarios concurrentes. En sus peores casos, MongoDB cayó 94 % y Redis 95 %.
Ubicación: Sección *Multi-regions deployment*, apartado *Throughput in writing operations*, p. 11.

Nota personal

Me llamó la atención que en una sola región Cassandra aguantaba bien las escrituras y que la caída fuerte apareció recién al separar las réplicas por regiones. Eso me hizo ver que en el TA-IND-01 tengo que tratar por separado el nivel de consistencia y la ubicación de los nodos, porque mezclados no se entiende cuál de los dos causa la pérdida. Voy a usar este dato en la sección sobre replicación geográfica.

Ficha 3: Consistencia y compromisos en bases de datos replicadas

Referencia (IEEE): J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko, and A. Sheikh-Akbari, "Consistency issue and related trade-offs in distributed replicated systems and databases: A review," *Radioelectronic and Computer Systems*, no. 2, pp. 171–179, 2023. DOI: 10.32620/reks.2023.2.14

Identificador: DOI: 10.32620/reks.2023.2.14

Tipo de fuente: Artículo de revisión en revista de acceso abierto indexada en Scopus y DOAJ (Radioelectronic and Computer Systems)

Resumen

Es un artículo de revisión sobre los compromisos que aparecen al replicar una base de datos en varios nodos, que además clasifica trabajos recientes sobre almacenamiento distribuido. Los autores parten de los teoremas CAP y PACELC, que explican el choque

entre consistencia y disponibilidad o latencia, y suman otros pares, como durabilidad contra rendimiento o disponibilidad contra costo y consumo de energía. Después agrupan los artículos revisados por temas usando nubes de palabras y una clasificación propia. Encuentran que la consistencia y la replicación dominan la investigación, y que crece el interés por modelos híbridos basados en consistencia causal.

Dato concreto

Según PACELC, si hay una partición de red la base elige entre disponibilidad y consistencia, y si no la hay elige entre latencia y consistencia. Con esa regla, los autores ubican a Cassandra como PA/EL (prefiere disponibilidad y baja latencia) y a MongoDB como PA/EC (prefiere disponibilidad ante una partición, pero consistencia cuando la red funciona normal).
Ubicación: Sección 1.2 (PACELC Theorem), p. 172.

Nota personal

Conocía lo básico de CAP, pero no PACELC, y me parece más útil porque explica qué pasa cuando la red funciona bien, que es casi todo el tiempo. Me sorprendió que el mismo artículo ponga a MongoDB como CP en la lista de CAP y como PA/EC en la de PACELC; lo voy a comentar en el informe porque muestra que la etiqueta depende de cómo se configure la base. Esta ficha la uso en el marco teórico del TA-IND-01, antes de la comparación.

Ficha 4: Algoritmos de consenso tolerantes a fallos bizantinos

Referencia (IEEE): G. Zhang et al., "Reaching consensus in the Byzantine Empire: A comprehensive review of BFT consensus algorithms," *ACM Computing Surveys*, vol. 56, no. 5, 2024, Art. no. 134. DOI: 10.1145/3636553

Identificador: DOI: 10.1145/3636553

Tipo de fuente: Artículo de revista indexada (ACM Computing Surveys)

Resumen

Es una revisión sistemática de algoritmos de consenso tolerantes a fallos bizantinos (BFT), que permiten que un grupo de servidores se ponga de acuerdo aunque algunos se comporten de forma arbitraria o maliciosa. Con el auge de blockchain aparecieron tantos algoritmos nuevos que entenderlos se volvió difícil, porque cada uno parte de supuestos y métricas distintos. Proponen una taxonomía con tres categorías (más eficientes, más robustos y más disponibles) y comparan la complejidad de mensajes y de tiempo de cada uno. También describen cada protocolo paso a paso con diagramas. PBFT aparece como la base de casi todos, aunque su costo en mensajes crece cuadráticamente.

Dato concreto

Para tolerar f fallos benignos, protocolos como Paxos o Raft necesitan quórums de $f+1$ nodos. Con fallos bizantinos el quórum mínimo sube a $2f+1$ dentro de un total de $n = 3f+1$ nodos. Por ejemplo, aguantar un solo nodo malicioso exige al menos cuatro nodos.
Ubicación: Sección 2 (Background), párrafo *Byzantine quorums*, p. 134:5.

Nota personal

Antes pensaba que tolerar fallos era solo cuestión de tener réplicas, y con este artículo entendí que el tipo de fallo cambia cuántos nodos hacen falta. Cassandra y MongoDB están pensadas para fallos benignos, como la caída de un nodo, así que no voy a usar esta fuente para compararlas de forma directa. La uso en la sección de fundamentos del TA-IND-01 para explicar qué es el consenso y por qué los quórums de estas bases son más pequeños que los de un sistema BFT.

Ficha 5: Desempeño de un clúster Cassandra replicado en AWS

Referencia (IEEE): A. Gorbenko, A. Karpenko, and O. Tarasyuk, "Performance evaluation of various deployment scenarios of the 3-replicated Cassandra NoSQL cluster on AWS," *Radioelectronic and Computer Systems*, no. 4, pp. 157–165, 2021. DOI: 10.32620/reks.2021.4.13

Identificador: DOI: 10.32620/reks.2021.4.13

Tipo de fuente: Artículo de revista de acceso abierto indexada en Scopus y DOAJ (Radioelectronic and Computer Systems)

Resumen

Es un reporte experimental sobre el rendimiento de un clúster de Cassandra con tres réplicas montado en Amazon AWS. Los autores señalan que CAP y PACELC describen la relación entre consistencia y latencia solo de forma cualitativa, y quieren ponerle números. Probaron cuatro formas de ubicar al cliente y a las réplicas, en una sola región o repartidas entre regiones, usando YCSB con 100 a 1000 hilos y los niveles ONE, QUORUM y ALL. También proponen una notación para describir esos despliegues. Con todo en una región, la consistencia fuerte cuesta poco; cuando las réplicas están en regiones distintas, la latencia se dispara.

Dato concreto

En el escenario (a), con el cliente y las tres réplicas en la misma región, el throughput de lectura bajó de 16 830 ops/s con ONE a 13 735 ops/s con ALL, cerca de un 18 % menos. En el escenario (c), con dos réplicas en otras regiones, la lectura con ONE llegó a 15 160 ops/s y con ALL apenas a 355 ops/s.
Ubicación: Sección 2.2 y Tabla 2, p. 160.

Nota personal

Es la única de mis fuentes que estudia solo Cassandra, y además prueba QUORUM, un nivel intermedio que las fichas 1 y 2 no incluyen. Me sirvió ver que con ONE el despliegue multirregión rinde apenas un 13 % menos que el local, porque el cliente siempre manda las peticiones al nodo más cercano. Voy a usar la Tabla 2 en la sección del TA-IND-01 dedicada a Cassandra y como referencia si repito alguna prueba con YCSB.

Referencias

[1] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Benchmarking consistency levels of cloud-distributed NoSQL databases using YCSB," *IEEE Access*, vol. 13, pp. 63428–63438, 2025. DOI: 10.1109/ACCESS.2025.3558923

[2] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications," *Journal of Cloud Computing*, vol. 13, no. 1, 2024, Art. no. 158. DOI: 10.1186/s13677-024-00716-7

[3] J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko, and A. Sheikh-Akbari, "Consistency issue and related trade-offs in distributed replicated systems and databases: A review," *Radioelectronic and Computer Systems*, no. 2, pp. 171–179, 2023. DOI: 10.32620/reks.2023.2.14

[4] G. Zhang et al., "Reaching consensus in the Byzantine Empire: A comprehensive review of BFT consensus algorithms," *ACM Computing Surveys*, vol. 56, no. 5, 2024, Art. no. 134. DOI: 10.1145/3636553

[5] A. Gorbenko, A. Karpenko, and O. Tarasyuk, "Performance evaluation of various deployment scenarios of the 3-replicated Cassandra NoSQL cluster on AWS," *Radioelectronic and Computer Systems*, no. 4, pp. 157–165, 2021. DOI: 10.32620/reks.2021.4.13